

Practical Assignment

Advanced Topics in Databases

Election Analytics Platform for Portugal

Build a serious database-backed system for analysing Portuguese elections, with a strong focus on data modelling, ETL, PostgreSQL/PostGIS, analytical SQL, and a thin web frontend.

Group size	2 or 3 students
Submission deadline	4 June 2026, 23:59 (Moodle)
Oral presentations	5 June 2026
Project weight	40% of the final course grade
Oral presentation weight	10% of the final course grade

1. Context and aim

The goal of this assignment is to design and implement an advanced database system that can power a website for election analysis. **The focus of the work must be the database component.** The website is important, but it should be treated as a thin layer on top of a well-designed and well-implemented database platform.

The main inspiration for the type of system to build is the official Portuguese election results portal published by the Ministério da Administração Interna (MAI). *You are not expected to reproduce that site in full.* Instead, you should build a smaller but technically solid system that demonstrates good modelling, robust ETL, meaningful analytical queries, and clear presentation of results.

The assignment should combine:

- an operational relational database for election data;
- a data warehouse for analytical queries;
- ETL pipelines that load and clean official datasets;
- SQL programming using functions, PL/pgSQL, and triggers;
- spatial storage and visualization of Portuguese administrative territory;
- a simple web frontend developed in Python.

2. Learning objectives

- Design a normalized relational schema for an election information system.
- Design and populate a data warehouse suited to analytical and comparative election queries.
- Implement rerunnable ETL processes from official public datasets into PostgreSQL/PostGIS.
- Use SQL functions, PL/pgSQL, and triggers to simplify queries and improve data consistency.

- Write analytical SQL using the D'Hondt method, window functions, advanced aggregates, GROUP BY ROLLUP, and GROUP BY CUBE.
- Expose the data through a simple Python web application using psycopg2 and meaningful visualizations.

3. Recommended technical framework

Mandatory core technologies: PostgreSQL, PostGIS, Python, and psycopg2.

Recommended web framework: Flask. It is intentionally simple and keeps the focus on the database. Other lightweight frameworks may be used if justified, but the project must remain clearly database-centred.

Recommended mapping / charting tools: Leaflet for maps, and Matplotlib or Plotly for charts.

Important note: Do not hide the database work behind an ORM. SQL must be explicit, visible, and central to the project. Using SQLAlchemy only as a connection/helper layer is acceptable, but the core logic must remain in SQL and PL/pgSQL.

4. Official data sources

You should use official Portuguese data sources whenever possible. A recommended baseline for all groups is the 2021 local elections (Autárquicas 2021), but stronger groups may extend the project to other elections or historical comparisons.

- CNE official spreadsheet package for Autárquicas 2021: https://www.cne.pt/sites/default/files/dl/2021al_mapa_oficial.zip
- CNE page for Eleições Autárquicas 2021: <https://www.cne.pt/content/eleicoes-autarquicas-2021>
- DGT / CAOP official administrative boundaries page: <https://www.dgterritorio.gov.pt/atividades/cartografia/cartografia-tematica/caop>
- MAI election results website for inspiration: <https://www.eleicoes.mai.gov.pt/autarquicas2021/>

For the territorial component, you should use official CAOP boundaries. You may choose the 2021 version to match the election year or a more recent compatible version if you clearly explain and justify the choice.

5. Project requirements

5.1 Operational data model

- Design a normalized relational schema for election data.
- Your model should cover, at minimum, elections, offices/organs, territorial units, parties, coalitions, candidacies, vote results, seat results, and turnout-related information.
- Primary keys, foreign keys, uniqueness constraints, and indexing strategy must be justified.
- The schema should be sufficiently general to support territorial drill-down and analytical comparisons.

5.2 Staging area, ETL, and data warehouse

- Implement a rerunnable ETL pipeline in Python.
- Use a staging area/schema before loading the cleaned operational schema.

- Build a data warehouse schema suitable for analytical queries. This should include at least one fact table and multiple dimension tables.
- Document how you reconcile identifiers, names, missing values, and territorial mismatches between the election files and CAOP data.
- A substantial part of the work should be dedicated to the ETL process and to the modelling of the warehouse.

5.3 SQL functions, PL/pgSQL, and triggers

- Implement at least 3 SQL functions or views that simplify important query logic.
- Implement at least 2 PL/pgSQL routines (functions or procedures).
- Implement at least 2 triggers designed to improve consistency or enforce meaningful business rules.
- The use of functions and triggers must be purposeful. Avoid artificial examples that add complexity without improving the system.

Examples of good use cases: seat allocation by D'Hondt, turnout computation, vote share computation, consistency checks on mandates, maintenance of summary tables, or integrity rules involving coalitions and territorial hierarchies.

5.4 Analytical SQL requirements

- At least one implementation of the D'Hondt method for seat allocation.
- At least 3 analytical queries using window functions.
- At least 1 query using GROUP BY ROLLUP.
- At least 1 query using GROUP BY CUBE.
- At least 1 query using advanced aggregate features such as FILTER, STRING_AGG, ARRAY_AGG, JSON aggregation, or a similarly appropriate construct.
- The analytical layer should support comparisons across territories, parties, and/or elections.

5.5 Spatial data and visualization

- Load official administrative boundaries into PostGIS.
- Support at least district- and municipality-level visualization. Stronger groups may include parish-level detail.
- The frontend must include an interactive map or drill-down navigation based on the stored geometries.
- The system must generate charts from the database results using Python and a plotting library.

5.6 Web frontend

The frontend should be simple, clean, and functional. The goal is not to build a visually complex product, but to demonstrate that the database can power a useful election analysis website.

- Recommended approach: a Flask web application.
- Mandatory: the application must query PostgreSQL/PostGIS through Python using psycopg2.
- Mandatory: the user must be able to select an election and a territorial unit.
- Mandatory: the page must display a results table and at least two meaningful visual outputs.

- The map should interact with the rest of the interface, for example by selecting a territory and updating the results shown.

6. Scope, minimum expectations, and freedom to extend

Minimum acceptable scope: one real election dataset, a working PostgreSQL/PostGIS database, a warehouse, ETL scripts, the required SQL programming components, and a simple web frontend with map-based territorial exploration.

You are encouraged to extend the project if you wish. Possible extensions include historical comparisons, support for more than one election type, additional territory levels, materialized views, precomputed aggregates, or richer frontend interaction.

7. Deliverables

Submit one repository or archive per group on Moodle. A suggested structure is the following:

- /sql/ — DDL for the operational schema, warehouse schema, functions, views, triggers, and indexes;
- /etl/ — ETL scripts, configuration, and instructions;
- /app/ — the Python web application;
- /docs/ — report, ER diagrams, screenshots, and any supporting material;
- /slides/ — the oral presentation slides;
- README.md — clear instructions to reproduce the project.

Important: If the raw official datasets are too large to include, provide download instructions and/or a script that retrieves them automatically.

7.1 Report

Include a short report of approximately 5 to 8 pages (excluding appendices). The report should be concise, technical, and well structured.

- Problem scope and chosen election datasets;
- Operational schema and warehouse design rationale;
- ETL pipeline and data cleaning decisions;
- Functions, PL/pgSQL routines, and triggers;
- Analytical queries and key findings;
- Frontend overview and visualizations;
- Known limitations and possible extensions;
- Short statement of the contribution of each group member.

7.2 Oral presentation

Oral presentations will take place on 5 June 2026. Each group should prepare a short presentation with slides demonstrating the architecture, database design, ETL pipeline, analytical SQL, and the working frontend.

- All students must be able to explain the project and answer questions about the database design and implementation.
- The exact presentation schedule/order will be announced separately.
- The oral component is worth 10% of the final course grade.

8. Assessment

The practical assignment is worth 40% of the final course grade, and the oral presentation is worth 10%. Within the practical assignment itself, the work will be evaluated approximately according to the following rubric:

Project component	Weight inside project
Data modelling: operational schema, keys, constraints, and indexing strategy	20%
ETL pipeline and data warehouse design	20%
Functions, PL/pgSQL, triggers, and consistency mechanisms	20%
Analytical SQL: D'Hondt, window functions, ROLLUP, CUBE, and advanced aggregates	20%
Frontend, spatial visualization, and integration with the database	10%
Documentation, reproducibility, and overall technical quality	10%

9. Good practice guidelines

- Start by validating a small subset of the data, then scale up.
- Prefer stable identifiers to free text whenever the source data provides them.
- Document all modelling assumptions and all ETL transformations clearly.
- Keep the frontend simple and invest most of the effort in the database and ETL components.
- Test your analytical queries carefully against official results.
- Remember that a polished interface is not a substitute for a well-designed database.

10. Academic integrity and teamwork

- The assignment must be completed in groups of 2 or 3 students.
- External code snippets, tutorials, AI tools, or libraries may be used only if their role is transparent and all submitted work is fully understood by the group.
- You must cite external resources when appropriate.
- All group members are expected to understand the schema, ETL logic, analytical SQL, and frontend integration.